

Destructuring Assignment - Simple Hinglish Summary

Source: <https://javascript.info/destructuring-assignment>

Main baat

JS ke do sabse zyada use hone wale data structures **Object** aur **Array** hain.

- Object: data items ko key ke saath ek entity me rakhta hai
- Array: data items ko ordered list me rakhta hai

Lekin jab inhe function me pass karte hain, to hume sab kuch nahi chahiye hota. Function ko sirf kuch elements ya properties chahiye.

Destructuring assignment ek special syntax hai jisse arrays ya objects ko "**unpack**" karke kai **variables** me baant sakte hain. Complex functions (kai parameters, default values) ke saath bhi ye bahut achha chalta hai.

1. Array destructuring

```
javascript

// naam aur surname wala array
let arr = ["John", "Smith"]

// destructuring assignment
// firstName = arr[0]
// surname = arr[1]
let [firstName, surname] = arr;

alert(firstName); // John
alert(surname);  // Smith
```

Ab array members ki jagah variables se kaam kar sakte hain.

`split` ya array return karne wale dusre methods ke saath bahut achha lagta hai:

```
javascript

let [firstName, surname] = "John Smith".split(' ');
alert(firstName); // John
alert(surname);  // Smith
```

"Destructuring" ka matlab "destructive" nahi hai. Ye items ko variables me **copy** karta hai, array khud nahi badalta. Ye sirf is ka chhota tarika hai:

javascript

```
let firstName = arr[0];  
let surname = arr[1];
```

Elements ignore karna (extra comma se)

javascript

```
// dusra element nahi chahiye  
let [firstName, , title] = ["Julius", "Caesar", "Consul", "of the Roman Republi  
  
alert( title ); // Consul
```

Dusra element skip hua, teesra (`title`) me gaya, aur baaki items bhi skip (kyunki unke liye variables nahi hain).

Right side par koi bhi iterable chalta hai

javascript

```
let [a, b, c] = "abc"; // ["a", "b", "c"]  
let [one, two, three] = new Set([1, 2, 3]);
```

Andar se destructuring right value par iterate karke kaam karta hai (`for..of`) ka syntax sugar).

Left side par koi bhi "assignable" chalta hai

Jaise object ki property:

javascript

```
let user = {};  
[user.name, user.surname] = "John Smith".split(' ');  
  
alert(user.name);    // John  
alert(user.surname); // Smith
```

`.entries()` ke saath loop

javascript

```
let user = {
  name: "John",
  age: 30
};

// keys-and-values par loop
for (let [key, value] of Object.entries(user)) {
  alert(`${key}:${value}`); // name:John, phir age:30
}
```

Map ke liye aur aasan, kyunki wo khud iterable hai:

javascript

```
let user = new Map();
user.set("name", "John");
user.set("age", "30");

// Map [key, value] pairs ki tarah iterate hota hai
for (let [key, value] of user) {
  alert(`${key}:${value}`); // name:John, phir age:30
}
```

Variables swap karne ki trick

javascript

```
let guest = "Jane";
let admin = "Pete";

// values swap karo: guest=Pete, admin=Jane
[guest, admin] = [admin, guest];

alert(`${guest} ${admin}`); // Pete Jane (swap ho gaya!)
```

Yahan do variables ka temporary array banakar turant ulte order me destructure kiya. Do se zyada variables bhi aise swap ho sakte hain.

Rest **...**

Array left ke list se lamba ho to "extra" items ignore ho jate hain:

javascript

```
let [name1, name2] = ["Julius", "Caesar", "Consul", "of the Roman Republic"];

alert(name1); // Julius
alert(name2); // Caesar
// baaki items kahin assign nahi hue
```

Baaki sab bhi chahiye to (...) ke saath ek aur variable jodo:

javascript

```
let [name1, name2, ...rest] = ["Julius", "Caesar", "Consul", "of the Roman Repu

// rest array hai, 3rd item se shuru
alert(rest[0]);      // Consul
alert(rest[1]);      // of the Roman Republic
alert(rest.length);  // 2
```

rest ki jagah koi bhi naam chalega, bas pehle teen dots ho aur wo destructuring me **sabse last** ho.

Default values

Array chhota ho to error nahi aata, missing values undefined hoti hain:

javascript

```
let [firstName, surname] = [];

alert(firstName); // undefined
alert(surname);   // undefined
```

Missing value ki jagah default chahiye to (=) se do:

javascript

```
// default values
let [name = "Guest", surname = "Anonymous"] = ["Julius"];

alert(name);    // Julius (array se)
alert(surname); // Anonymous (default use hua)
```

Default values complex expressions ya function calls bhi ho sakti hain. Wo **sirf tab evaluate** hoti hain jab value na mile:

javascript

```
// sirf surname ke liye prompt chalega  
let [name = prompt('name?'), surname = prompt('surname?')] = ["Julius"];
```

2. Object destructuring

Basic syntax:

javascript

```
let {var1, var2} = {var1:..., var2:...}
```

Right side par existing object hota hai jise variables me todna hai. Left side par object jaisa **"pattern"** hota hai. Sabse simple case me `{...}` me variable names ki list.

javascript

```
let options = {  
  title: "Menu",  
  width: 100,  
  height: 200  
};  
  
let {title, width, height} = options;  
  
alert(title); // Menu  
alert(width); // 100  
alert(height); // 200
```

Order matter nahi karta:

javascript

```
let {height, width, title} = { title: "Menu", height: 200, width: 100 }
```

Dusre naam ke variable me daalna (colon se)

`options.width` ko `w` naam ke variable me daalna ho:

javascript

```
// { sourceProperty: targetVariable }
let {width: w, height: h, title} = options;

// width -> w
// height -> h
// title -> title

alert(title); // Menu
alert(w);      // 100
alert(h);      // 200
```

Colon batata hai **"kya kahan jaye"**.

Default values

Missing properties ke liye `=` se default:

```
javascript

let options = {
  title: "Menu"
};

let {width = 100, height = 200, title} = options;

alert(title); // Menu
alert(width); // 100
alert(height); // 200
```

Default koi bhi expression ya function call ho sakta hai, jo tabhi evaluate hota hai jab value na mile.

Colon aur equal dono saath:

```
javascript

let {width: w = 100, height: h = 200, title} = options;

alert(title); // Menu
alert(w);      // 100
alert(h);      // 200
```

Sirf jo chahiye wahi nikalo:

```
javascript
```

```
let options = {
  title: "Menu",
  width: 100,
  height: 200
};

// sirf title nikala
let { title } = options;

alert(title); // Menu
```

Rest pattern (...) (...)

Object me variables se zyada properties hon aur baaki kahin assign karni hon:

```
javascript

let options = {
  title: "Menu",
  height: 200,
  width: 100
};

// title = title naam ki property
// rest = baaki properties ka object
let {title, ...rest} = options;

// ab title="Menu", rest={height: 200, width: 100}
alert(rest.height); // 200
alert(rest.width); // 100
```

Gotcha: let ke bina

Variables pehle se bane hon aur bina let ke destructure karna ho to ye **error** deta hai:

```
javascript

let title, width, height;

// is line me error
{title, width, height} = {title: "Menu", width: 200, height: 100};
```

Kyu? Main code flow me {...} ko JS **code block** maanta hai. **Fix:** पूरी expression ko **brackets** () me daalo:

javascript

```
let title, width, height;

// ab theek hai
({title, width, height} = {title: "Menu", width: 200, height: 100});

alert( title ); // Menu
```

3. Nested destructuring

Object ya array me aur objects/arrays ho to left side par waisa hi complex pattern bana kar deeper hisse nikal sakte hain:

javascript

```
let options = {
  size: {
    width: 100,
    height: 200
  },
  items: ["Cake", "Donut"],
  extra: true
};

// clarity ke liye kai lines me
let {
  size: { // size yahan daalo
    width,
    height
  },
  items: [item1, item2], // items yahan assign karo
  title = "Menu" // object me nahi hai (default use hoga)
} = options;

alert(title); // Menu
alert(width); // 100
alert(height); // 200
alert(item1); // Cake
alert(item2); // Donut
```

`options` ki saari properties jo left me hain wo variables me gayi (`extra` ko chhodkar).
Dhyan: `size` aur `items` ke liye variables nahi bane, kyunki hum unka content le rahe hain.

4. Smart function parameters

Kabhi function ke **bahut saare parameters** hote hain, jinme zyadatar optional. Khaaskar user interfaces me. Jaise menu banane wala function: width, height, title, items list, etc.

Bura tarika:

javascript

```
function showMenu(title = "Untitled", width = 200, height = 100, items = []) {  
  // ...  
}
```

Problem: arguments ka order yaad kaise rakhein? Aur jab zyadatar defaults theek hon to call kaise karein?

javascript

```
// jahan default theek hai wahan undefined  
showMenu("My Menu", undefined, undefined, ["Item1", "Item2"])
```

Ye bahut **bhaddha** hai, aur parameters badhne par padhna mushkil ho jata hai.

Destructuring isse bachata hai! Parameters object ki tarah pass karo, aur function turant use variables me tod de:

javascript

```
// function ko object pass karte hain  
let options = {  
  title: "My menu",  
  items: ["Item1", "Item2"]  
};  
  
// ...aur wo turant variables me expand karta hai  
function showMenu({title = "Untitled", width = 200, height = 100, items = []})  
  // title, items - options se aaye,  
  // width, height - defaults use hue  
  alert( `${title} ${width} ${height}` ); // My Menu 200 100  
  alert( items ); // Item1, Item2  
}  
  
showMenu(options);
```

Complex destructuring (nested aur colon mapping) bhi:

javascript

```
function showMenu({
  title = "Untitled",
  width: w = 100, // width w me jayega
  height: h = 200, // height h me jayega
  items: [item1, item2] // items ka pehla element item1, dusra item2
}) {
  alert( `${title} ${w} ${h}` ); // My Menu 100 200
  alert( item1 ); // Item1
  alert( item2 ); // Item2
}

showMenu(options);
```

Poori syntax destructuring assignment jaisa hi hai:

javascript

```
function({
  incomingProperty: varName = defaultValue
  ...
})
```

Dhyan: aisa destructuring maanta hai ki `showMenu()` ko **argument mila hai**. Saare values default chahiye to **khali object** `{}` dena padega:

javascript

```
showMenu({}); // theek hai, sab default

showMenu(); // ye error dega
```

Fix: poore parameters object ki default value `{}` bana do:

javascript

```
function showMenu({ title = "Menu", width = 100, height = 200 } = {}) {
  alert( `${title} ${width} ${height}` );
}

showMenu(); // Menu 100 200
```

Summary

Destructuring assignment object ya array ko turant kai variables me map karne deta hai.

Poora object syntax:

```
javascript  
  
let {prop : varName = defaultValue, ...rest} = object
```

Matlab property `(prop)` variable `(varName)` me jaye, aur agar wo property na ho to `(defaultValue)` value use ho. Jin properties ka mapping nahi hai wo `(rest)` object me copy hoti hain.

Poora array syntax:

```
javascript  
  
let [item1 = defaultValue, item2, ...rest] = array
```

Pehla item `(item1)` me, dusra `(item2)` me, aur baaki sab `(rest)` array banta hai.

Nested arrays/objects se data nikalne ke liye left side ka structure right side jaisa hona chahiye.

Practice Tasks (Answers)

1. Destructuring assignment: Object `(user = { name: "John", years: 30 })` se:

- `(name)` property → variable `(name)`
- `(years)` property → variable `(age)`
- `(isAdmin)` property → variable `(isAdmin)` (na ho to `(false)`)

```
javascript  
  
let user = {  
  name: "John",  
  years: 30  
};  
  
let {name, years: age, isAdmin = false} = user;  
  
alert( name );    // John  
alert( age );     // 30  
alert( isAdmin ); // false
```

2. **Sabse zyada salary** (**topSalary**): **salaries** object me se sabse zyada kamane wale ka naam return karo. Khali ho to **null**.

javascript

```
function topSalary(salaries) {  
  
    let maxSalary = 0;  
    let maxName = null;  
  
    for(const [name, salary] of Object.entries(salaries)) {  
        if (maxSalary < salary) {  
            maxSalary = salary;  
            maxName = name;  
        }  
    }  
  
    return maxName;  
}
```

Quick Summary

Kaam	Syntax
Array se variables	<code>let [a, b] = arr;</code>
Element skip	<code>let [a, , c] = arr;</code>
Baaki sab ek array me	<code>let [a, ...rest] = arr;</code>
Default value	<code>let [a = 1, b = 2] = arr;</code>
Object se variables	<code>let {title, width} = obj;</code>
Dusre naam me	<code>let {width: w} = obj;</code>
Default ke saath	<code>let {width = 100} = obj;</code>
Naam + default	<code>let {width: w = 100} = obj;</code>
Object ka rest	<code>let {title, ...rest} = obj;</code>
Nested	<code>let {size: {width}, items: [a, b]} = obj;</code>
Swap	<code>[a, b] = [b, a];</code>
Bina <code>let</code> ke object	<code>({a, b} = obj);</code> (brackets zaruri)
Function parameters	<code>function f({a = 1, b = 2} = {}) {...}</code>
Object par loop	<code>for (let [k, v] of Object.entries(obj))</code>

Yaad rakho:

- Destructuring original ko **badalta nahi**, sirf copy karta hai.
- Right side par koi bhi **iterable** chalta hai (array destructuring me).
- `...rest` hamesha **aakhri** hona chahiye.
- Bina `let` ke object destructuring me poori line `( )` me likho.